Polynomial Curve Fitting (Part 1)

Peter Adepoju
(My study note on the topic)

1 The Regression Problem

Regression analysis is foundational in statistical learning and data science. It is a method for exploring relationships between variables. Specifically, we seek to model the relationship between a dependent variable (often called the *target* or *label*, denoted as t) and one or more independent variables (often called the *input*, *features*, or *regressors*, denoted as x).

In this section, we will not begin with complex real-world data, which often includes hidden biases and unknown underlying processes. Instead, we will construct a *synthetic* problem. By generating data ourselves, we gain access to the "ground truth"—the actual mathematical function governing the data. This allows us to precisely measure how well our models perform, a luxury we rarely have in real-world scenarios (Bishop, 2006).

1.1 Introduction to Curve Fitting

Our goal is to simulate a supervised learning scenario. We assume there is an underlying physical process that generates data, but this process is corrupted by random noise. We wish to discover the original process given only the noisy observations.

1.1.1 The Mathematical Foundation

We begin by defining the underlying "true" function. For this example, and following the classic formulation by (Bishop, 2006), we choose the sine function. Let the input variable x be a scalar value. We define the deterministic function $h(x)$ as:

$$h(x) = \sin(2\pi x) \tag{1}$$

Why $\sin(2\pi x)$? The standard sine function, $\sin(x)$, has a period of 2π. By multiplying the argument by 2π, we normalize the period to exactly 1. This means the wave completes a full cycle as x moves from 0 to 1. This normalization makes the mathematics cleaner and allows us to focus our analysis on the interval $x \in [0, 1]$. However, in the real world, data is rarely perfect. Sensors have inaccuracies, and unobserved variables influence outcomes. To simulate this, we add a noise term, ϵ, to our target values. Our observed target t becomes:

$$t = h(x) + \epsilon = \sin(2\pi x) + \epsilon \tag{2}$$

Here, ϵ is a random variable. A standard assumption in regression analysis is that this noise follows a **Gaussian (Normal) distribution** with a mean of zero and a variance of σ^2 (written as $\mathcal{N}(0, \sigma^2)$).

Why Gaussian Noise? The Central Limit Theorem suggests that when many small, independent random factors influence a measurement, their sum tends to follow a Gaussian distribution. This makes Gaussian noise a physically realistic and mathematically convenient assumption for regression problems (Murphy, 2012).

1.1.2 Python Implementation

Now, we translate this mathematical definition into Python code.

```python
import numpy as np
import matplotlib.pyplot as plt

# 1. Set the random seed for reproducibility
np.random.seed(42)

# 2. Generate the "True" function curve
x_true = np.linspace(0, 1, 100)
t_true = np.sin(2 * np.pi * x_true)

# 3. Generate the Training Data (Observations)
N = 10  # Number of data points
x_train = np.linspace(0, 1, N)
# Add noise to the true function
noise = np.random.normal(scale=0.15, size=N)
t_train = np.sin(2 * np.pi * x_train) + noise

# 4. Visualization
plt.plot(x_true, t_true, 'g-', label="True Function $\sin(2\pi x)$")
plt.scatter(x_train, t_train, facecolor="none", edgecolor="b", s=50, label=
    "Noisy Observations")
plt.legend()
plt.show()
```

Listing 1: Generating Synthetic Data

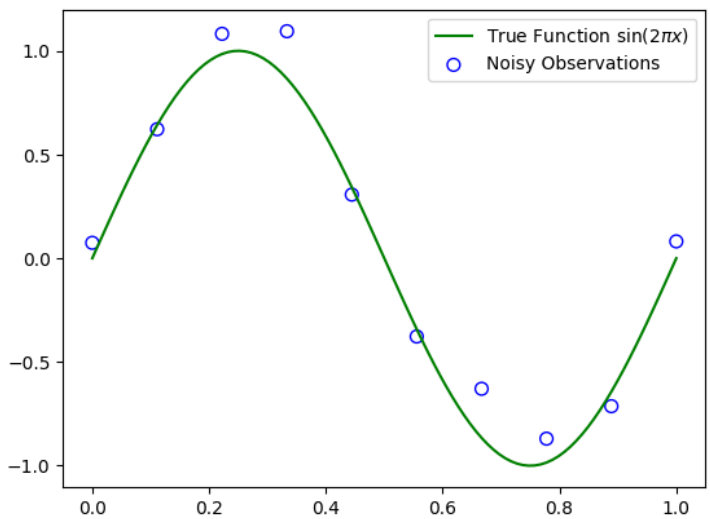


Figure 1: The green curve represents the true generating function $h(x) = \sin(2\pi x)$, while the blue circles represent $N = 10$ observations distorted by Gaussian noise.

1.1.3 Code Analysis

Line 5: `np.random.seed(42)`
In scientific computing, reproducibility is paramount. Computers generate "pseudorandom" numbers, which are deterministic sequences initialized by a "seed." By setting this seed to a specific integer (like 42), we ensure that every time we run this code, the "random" noise will be identical. This allows us to debug our model without the data changing under our feet.

Line 8: `np.linspace(0, 1, 100)`
Discretization: In mathematics, a real number line is continuous; there are infinite points between 0 and 1. Computers, however, are discrete systems. We cannot represent infinite points. The function 'linspace' (linear space) generates a sequence of 100 evenly spaced points between 0 and 1. We use 100 points here to create a visual illusion of a smooth curve when plotting.

Line 9: `np.sin(2 * np.pi * x_true)`
Vectorization: Note that `x_true` is a NumPy array containing 100 numbers. When we multiply it by `2 * np.pi` and pass it into `np.sin`, Python does **not** need a `for` loop to calculate the sine of each number individually. Instead, NumPy applies the operation element-wise to the entire array simultaneously. This is called **vectorization**. It is orders of magnitude faster than Python loops and mimics the mathematical notation where we apply a function f to a vector $\mathbf{x}$ (VanderPlas, 2016).

Line 15: `np.random.normal(...)`
This generates the noise term ϵ. The 'scale' parameter corresponds to the standard deviation σ. We chose $\sigma = 0.15$, meaning the observations will deviate from the true sine wave by an average of 0.15 units.

1.1.4 The Goal of Regression

Having generated this data, we can now formally state the problem. We are given the blue circles (the noisy training data $\mathbf{x}_{train}, \mathbf{t}_{train}$) **only**. We do not know the green line (the true function).

Our goal is to construct a model $y(x, \mathbf{w})$ such that for any new input value $\hat{x}$, the model predicts a target $\hat{t}$ that is close to the true (unknown) value $\sin(2\pi\hat{x})$. This distinguishes *curve fitting* from simple *interpolation*. We do not want to just connect the dots of the training data; we want to discover the underlying trend (Hastie et al., 2009).

1.2 The Nature of Data

In the context of the *Supervised Learning* paradigm, data is invariably split into two distinct categories: the information we *have* (Inputs) and the information we want to *predict* (Targets).

1.2.1 Terminology and Notation

Throughout this text, we will adhere to a consistent notational standard, widely used in machine learning literature (Bishop, 2006; Murphy, 2012).

1. **The Input Variable (x):** The input variable represents the information observed or measured. In our sine wave example, x represents the position along the horizontal axis. In a real-world scenario, such as predicting CO_2 levels, x might represent the time (year).

- **Synonyms:** Independent variable, feature, regressor, covariate, predictor.
- **Notation:** A single observation is denoted by scalar x. If we have multiple observations, we index them with n. Thus, x_n is the n-th input data point.
- **Vector Space:** In our simple example, x is a single number (scalar). However, in complex problems, an input might consist of multiple features (e.g., temperature, pressure, and humidity). In such cases, the input is a vector $\mathbf{x} \in \mathbb{R}^D$, where D is the dimensionality of the feature space.

2. **The Target Variable (t):** The target variable is the "ground truth" or the value we wish to predict. It is the value associated with a specific input x.

   - **Synonyms:** Dependent variable, label, response variable, outcome.
   - **Notation:** We denote the target value corresponding to the n-th input x_n as t_n.
   - **Crucial Distinction (t vs. y):** We strictly distinguish between the *observed* target value t (which comes from the dataset and includes noise) and the *predicted* value y (which comes from our model). Our goal is to make y as close to t as possible.

1.2.2 Data Representation in Python

One of the most frequent sources of error in scientific programming is the mismatch of array shapes. Mathematical vectors are strictly defined, but computer memory is linear. We must explicitly tell Python how to treat our data structure.

The Training Set: We denote the entire set of N observations as a collection of input-target pairs:

$$\mathcal{D} = \{(x_1, t_1), (x_2, t_2), \ldots, (x_N, t_N)\} \tag{3}$$

In our code from Section 1.1, we generated `x_train` and `t_train`. Let us inspect their structure.

```python
# Inspecting the shape of our generated data
print(f"Shape of x_train: {x_train.shape}")
print(f"Shape of t_train: {t_train.shape}")

# Output:
# Shape of x_train: (10,)
# Shape of t_train: (10,)
```

Listing 2: Inspecting Data Shapes

> In NumPy, a shape of `(10,)` indicates a **Rank-1 Array**. It is neither a row vector nor a column vector in the strict linear algebra sense; it is simply a flat list of numbers.

While Rank-1 arrays are sufficient for simple plotting, linear algebra operations require strict dimensions. We often need to treat our inputs as a **Column Vector**.

Mathematically, we represent the global input vector $\mathbf{x}$ as:

$$\mathbf{x} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_N \end{bmatrix} \tag{4}$$

This is an $N \times 1$ matrix. To convert a NumPy Rank-1 array into this matrix format, we use *reshaping*.

```python
# Converting flat array to column vector (Matrix of size N x 1)
x_vector = x_train.reshape(-1, 1)

print(f"New shape: {x_vector.shape}")
# Output: New shape: (10, 1)
```

Listing 3: Reshaping for Linear Algebra

The -1 Trick in Reshaping: In `x_train.reshape(-1, 1)`, the `-1` acts as a "wildcard." It tells NumPy: "I want 1 column, and you figure out how many rows are needed to make the data fit." Since we have 10 elements, NumPy calculates that we need 10 rows. This makes the code robust; it works regardless of whether $N = 10$ or $N = 1000$ (VanderPlas, 2016).

1.3 Why Polynomials?

Why is a polynomial a good choice for an unknown function like $\sin(2\pi x)$? The answer lies in **Taylor's Theorem**. In calculus, any smooth, continuous function can be approximated arbitrarily well by a polynomial, provided the degree M is sufficiently high. A polynomial of degree M is defined as:

$$y(x, \mathbf{w}) = w_0 + w_1 x + w_2 x^2 + \cdots + w_M x^M = \sum_{j=0}^{M} w_j x^j \tag{5}$$

The coefficients $w_0, \ldots, w_M$ are collectively denoted by the vector $\mathbf{w}$.

1.3.1 The Parameters (w)

The specific shape of the polynomial is determined by the coefficients $\mathbf{w} = (w_0, w_1, \ldots, w_M)^T$.

- w_0 is the **bias** or intercept (where the curve hits the y-axis).
- w_1 controls the linear slope.
- w_2 controls the curvature (concavity).
- w_j controls the influence of the j-th power of x.

Our goal in "learning" is simply to find the specific numbers in vector $\mathbf{w}$ that make the curve pass through our data points.

1.3.2 Linearity in Parameters vs. Non-Linearity in Inputs

Looking at Equation 5, the function $y(x, \mathbf{w})$ represents a **non-linear relationship** between x and y. If you plot y against x, you get a curve, not a line. However, the function is **linear** with respect to the coefficients $\mathbf{w}$.

Definition of a Linear Model: A model is considered "linear" if the output is a weighted sum of the parameters.

$$y(x, \mathbf{w}) = f(x; \mathbf{w}) \tag{6}$$

is linear *in the parameters* if:

$$\frac{\partial y}{\partial w_j} = \text{function of } x \text{ (independent of } \mathbf{w}) \tag{7}$$

Let us compare two models to verify this:

Model A (Polynomial): $y = w_0 + w_1 x + w_2 x^2$

- The derivative with respect to w_2 is x^2.
- This does not depend on w.
- Therefore, Model A is **Linear in Parameters**.

Model B (Exponential): $y = w_0 + x^{w_1}$

- Here, the parameter w_1 is in the exponent.
- The relationship between w_1 and y is complex and non-linear.
- Therefore, Model B is **Non-Linear in Parameters**.

Why do we care? Because our polynomial model is linear in terms of $\mathbf{w}$, the *error surface* (the landscape of mistakes we are trying to minimize) is a simple bowl shape (a convex parabola). This guarantees that there is a **single, unique best solution** for $\mathbf{w}$, which we can find using standard Linear Algebra (Strang, 2006).

If the model were non-linear in $\mathbf{w}$ (like neural networks), the error surface would be rugged with many peaks and valleys, making it much harder to find the best solution.

Thus, polynomial regression is technically a form of **Linear Regression**, even though it fits curved lines. We are fitting a linear combination of non-linear *basis functions* $(1, x, x^2, \dots)$ (Bishop, 2006).

2 Data Acquisition and Handling

In Section 1, we worked with a "toy" dataset. We defined the ground truth function $h(x) = \sin(2\pi x)$ and added artificial noise. In the real world, however, we are never given the ground truth. We are only given the noisy observations, and the underlying function is a mystery we must solve. To demonstrate this, we will use one of the most famous datasets in the history of environmental science: the **Mauna Loa Atmospheric CO_2 Records**.

2.1 Real-World Data Challenges

2.1.1 The Keeling Curve

Since 1958, C. David Keeling and the Scripps Institution of Oceanography have been measuring carbon dioxide levels at the Mauna Loa Observatory in Hawaii. This dataset is crucial for understanding climate change (Keeling et al., 1976).

Unlike our synthetic sine wave, this data presents several challenges distinct to real-world modeling:

1. **Unknown Function:** There is no equation given to us. We see a clear long-term upward trend (due to fossil fuel combustion) combined with a cyclic "sawtooth" pattern (due to seasonal vegetation cycles in the Northern Hemisphere).

2. **Messy Format:** Real data rarely comes in clean arrays. It often arrives in text files (`.txt`, `.csv`) containing metadata, comments, missing values, and inconsistent headers.

3. **Scale:** The input variable (x) is time (years like 1958.2027), and the target (t) is concentration (ppm like 315.71). These numbers are fundamentally different in magnitude from our previous range of $[0, 1]$.

Our objective remains the same: fit a polynomial curve to this data to capture the long-term trend.

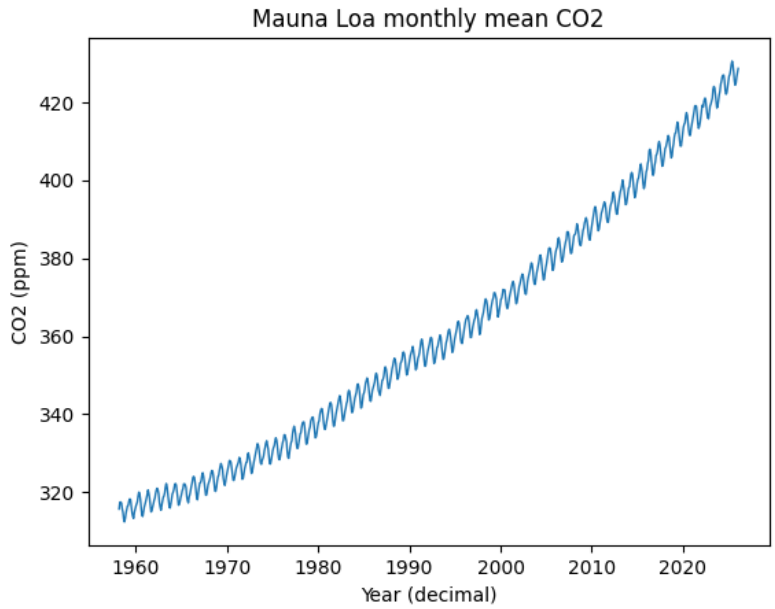


Figure 2: Visualization of the Mauna Loa monthly mean CO_2 measurements, historically known as the Keeling Curve.

2.2 Python Data Structures

A common question for beginners is: *Why do we need special libraries like Pandas and NumPy? Why not just use Python's built-in lists?* The answer lies in **memory architecture** and **computational efficiency**.

2.2.1 The Problem with Python Lists

A standard Python list (e.g., `data = [1.0, 2.0, 3.0]`) is essentially an array of *pointers* to Python objects.

- Each number is a full Python object containing reference counts, type information, and the value itself. The list stores addresses, not values.
- To perform math (e.g., `data * 2`), the CPU must traverse these pointers, check the type of every single object, and then execute the operation. This is computationally expensive (slow) and memory-inefficient.

2.2.2 NumPy and Pandas

To perform regression on datasets with hundreds or millions of points, we need **Vectorization**.

NumPy (Numerical Python): NumPy provides the `ndarray` object. Unlike a Python list, a NumPy array stores data in a *contiguous block of memory*, similar to C or Fortran (Harris et al., 2020).

- **Homogeneity:** All elements must be of the same type (e.g., `float64`).
- **SIMD Operations:** Because the data is packed together in memory, the CPU can use Single Instruction, Multiple Data (SIMD) instructions to process multiple numbers simultaneously.
- **No Overhead:** There are no pointers or type-checking loops during calculation.

Pandas (Python Data Analysis Library): While NumPy is excellent for *math*, it is poor for *data management*. Real data has labels (column names like "Year", "CO2") and indices. Pandas is built on top of NumPy (McKinney, 2010). It provides the `DataFrame`, which combines the speed of NumPy arrays and the usability of spreadsheets/SQL (for labels, handling missing data, and file parsing).

2.3 Reading and Cleaning Data

The Mauna Loa dataset is provided as a text file (`.txt`). Before we can fit a polynomial to it, we must parse this file to separate the numerical signal from the textual noise.

2.3.1 The Raw File Structure

Let us first inspect the raw content of the file.

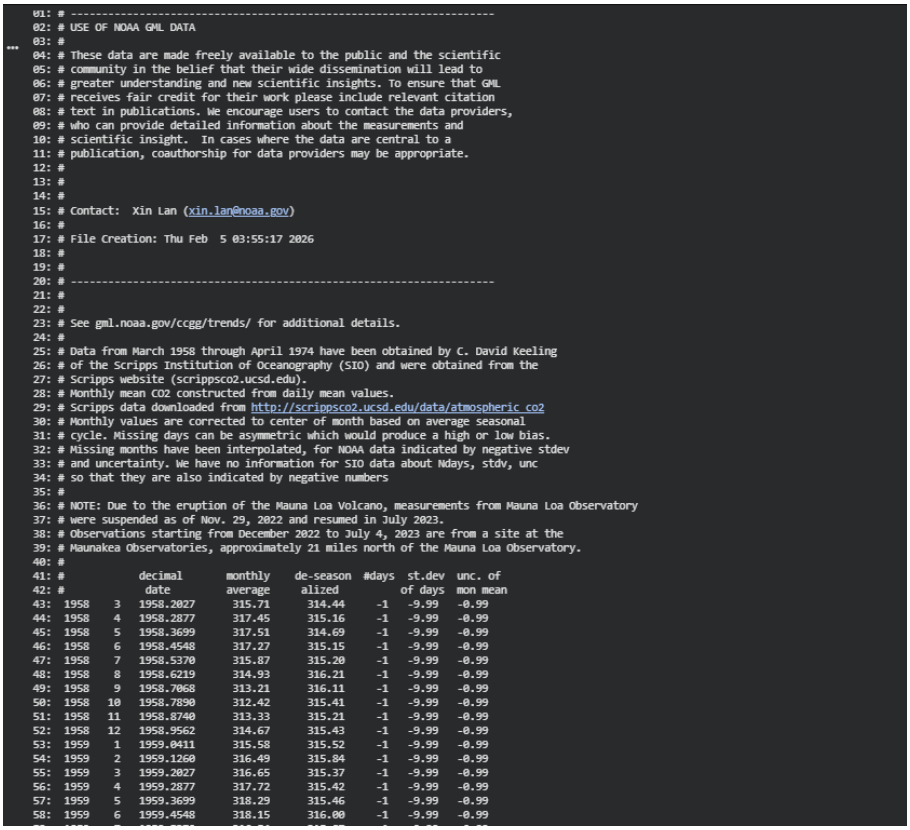
```python
import requests

url = "https://gml.noaa.gov/webdata/ccgg/trends/co2/co2_mm_mlo.txt"
txt = requests.get(url).text
lines = txt.splitlines()

# Print first non-comment lines
for ln in lines[:60]:
    if not ln.strip().startswith("#"):
        print(ln)
```

Listing 4: Inspecting the Raw Text

Analysis of the Raw Data: Upon inspection, the file reveals two distinct sections:

1. **Header/Metadata:** Dozens of lines starting with the hash character (#). These contain legal notes, column descriptions, and citations.
2. **Data Table:** Rows of numbers separated by varying amounts of whitespace.

If we were to blindly load this file into a matrix, the text strings (like "NOAA") would cause type errors, as they cannot be converted to floating-point numbers.



```
01: # ----------------------------------------------------------------------
02: # USE OF NOAA GML DATA
03: #
04: # These data are made freely available to the public and the scientific
05: # community in the belief that their wide dissemination will lead to
06: # greater understanding and new scientific insights. To ensure that GML
07: # receives fair credit for their work please include relevant citation
08: # text in publications. We encourage users to contact the data providers,
09: # who can provide detailed information about the measurements and
10: # scientific insight.  In cases where the data are central to a
11: # publication, coauthorship for data providers may be appropriate.
12: #
13: #
14: #
15: # Contact:  Xin Lan (xin.lan@noaa.gov)
16: #
17: # File Creation: Thu Feb  5 03:55:17 2026
18: #
19: #
20: # ----------------------------------------------------------------------
21: #
22: #
23: # See gml.noaa.gov/ccgg/trends/ for additional details.
24: #
25: # Data from March 1958 through April 1974 have been obtained by C. David Keeling
26: # of the Scripps Institution of Oceanography (SIO) and were obtained from the
27: # Scripps website (scrippsco2.ucsd.edu).
28: # Monthly mean CO2 constructed from daily mean values.
29: # Scripps data downloaded from http://scrippsco2.ucsd.edu/data/atmospheric_co2
30: # Monthly values are corrected to center of month based on average seasonal
31: # cycle. Missing days can be asymmetric which would produce a high or low bias.
32: # Missing months have been interpolated, for NOAA data indicated by negative stdev
33: # and uncertainty. We have no information for SIO data about Ndays, stdv, unc
34: # so that they are also indicated by negative numbers
35: #
36: # NOTE: Due to the eruption of the Mauna Loa Volcano, measurements from Mauna Loa Observatory
37: # were suspended as of Nov. 29, 2022 and resumed in July 2023.
38: # Observations starting from December 2022 to July 4, 2023 are from a site at the
39: # Maunakea Observatories, approximately 21 miles north of the Mauna Loa Observatory.
40: #
41: #            decimal       monthly    de-season  #days  st.dev  unc. of
42: #             date         average     alized          of days  mon mean
43:  1958    3   1958.2027      315.71      314.44     -1   -9.99   -0.99
44:  1958    4   1958.2877      317.45      315.16     -1   -9.99   -0.99
45:  1958    5   1958.3699      317.51      314.69     -1   -9.99   -0.99
46:  1958    6   1958.4548      317.27      315.15     -1   -9.99   -0.99
47:  1958    7   1958.5370      315.87      315.20     -1   -9.99   -0.99
48:  1958    8   1958.6219      314.93      316.21     -1   -9.99   -0.99
49:  1958    9   1958.7068      313.21      316.11     -1   -9.99   -0.99
50:  1958   10   1958.7890      312.42      315.41     -1   -9.99   -0.99
51:  1958   11   1958.8740      313.33      315.21     -1   -9.99   -0.99
52:  1958   12   1958.9562      314.67      315.43     -1   -9.99   -0.99
53:  1959    1   1959.0411      315.58      315.52     -1   -9.99   -0.99
54:  1959    2   1959.1260      316.49      315.84     -1   -9.99   -0.99
55:  1959    3   1959.2027      316.65      315.37     -1   -9.99   -0.99
56:  1959    4   1959.2877      317.72      315.42     -1   -9.99   -0.99
57:  1959    5   1959.3699      318.29      315.46     -1   -9.99   -0.99
58:  1959    6   1959.4548      318.15      316.00     -1   -9.99   -0.99
```

Figure 3: Snapshot of the raw content from the Mauna Loa CO_2 dataset file.

2.3.2 The Pandas Solution

While we could write a complex loop to parse lines manually, the Pandas library provides a highly optimized function, `read_csv`, which can handle these irregularities via specific arguments.

```python
import pandas as pd

df = pd.read_csv(
    url,
    comment='#',
    sep=r'\s+',
    header=None)

print(df.head(10))
```

Listing 5: Loading the Data with Pandas

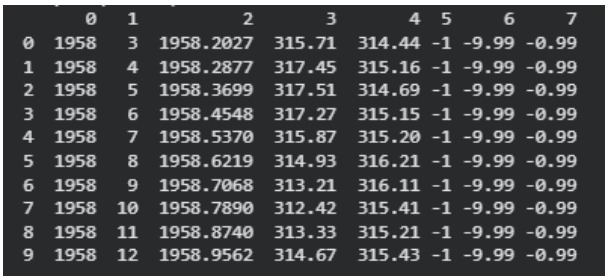
Let us break down *why* each argument is necessary.

1. The Comment Argument (`comment='#'`) The `read_csv` function parses the file line by line. By setting `comment='#'`, we instruct the parser to ignore any character following a hash mark, and ignore lines that begin with it entirely.

2. The Separator Argument (`sep=r'\s+'`) Standard CSV files use commas to separate values (e.g., `1958,315.71`). However, scientific text files often use whitespace to align columns visually for humans.

- **The Problem:** If we used a single space as a separator (`sep=' '`), multiple spaces would be interpreted as empty columns, filling our data with `NaN` (Not a Number) values.
- **The Solution:** We use the regular expression `\s+`. This tells Pandas: "Treat any chunk of whitespace, whether it is one space or ten spaces, as a single separator" (Friedl, 2006).

3. The Header Argument (`header=None`) The original file does not contain a standard single-row CSV header like `year,month,decimal,average`. Instead, the column names are buried inside the commented lines. Since the `comment='#'` argument strips those lines away, the first line Pandas sees is the first row of numbers: `1958 3 1958.2027 ....` If we do not specify 'header=None', Pandas will mistakenly assume this first row of data contains the names of the columns.



	0	1	2	3	4	5	6	7
0	1958	3	1958.2027	315.71	314.44	-1	-9.99	-0.99
1	1958	4	1958.2877	317.45	315.16	-1	-9.99	-0.99
2	1958	5	1958.3699	317.51	314.69	-1	-9.99	-0.99
3	1958	6	1958.4548	317.27	315.15	-1	-9.99	-0.99
4	1958	7	1958.5370	315.87	315.20	-1	-9.99	-0.99
5	1958	8	1958.6219	314.93	316.21	-1	-9.99	-0.99
6	1958	9	1958.7068	313.21	316.11	-1	-9.99	-0.99
7	1958	10	1958.7890	312.42	315.41	-1	-9.99	-0.99
8	1958	11	1958.8740	313.33	315.21	-1	-9.99	-0.99
9	1958	12	1958.9562	314.67	315.43	-1	-9.99	-0.99

Figure 4: Output of the `pd.read_csv` command showing the first 10 rows of the parsed data.

2.3.3 Renaming Columns

Because we stripped the headers, our DataFrame currently has meaningless column names (0–7). We must manually map these to the definitions provided in the file's metadata.

```
df.columns = ["year", "month", "decimal_date", "average",
                "deseasonalized", "ndays", "sdev", "unc"]
```

Listing 6: Renaming Columns

This step assigns semantic meaning to the numerical matrix, allowing us to access data via `df['average']` rather than `df[3]`.

2.4 Data Inspection

Before performing any mathematical operations, we must validate the integrity of our dataframe. This process, often called *Exploratory Data Analysis* (EDA), is crucial for identifying parsing errors, missing values, or physical impossibilities in the data (Tukey, 1977).

2.4.1 Verifying Data Types (`dtypes`)

The first check is technical: did Pandas interpret the bits and bytes correctly? We use the `df.dtypes` attribute to inspect the data type of each column.

```
print(df.dtypes)

# Output:
# year                int64
# month               int64
```

```
# decimal_date     float64
# average          float64
# deseasonalized   float64
# ndays              int64
# sdev             float64
# unc              float64
# dtype: object
```

Listing 7: Checking Data Types

Why this matters:
In Python, mathematical operations are type-dependent.

- If a column is `float64` or `int64`, NumPy can perform vector algebra on it.
- If a column is `object` (usually indicating strings), an operation like `df['average'] + 1` might either crash or perform string concatenation (turning "315" into "3151") instead of addition.

2.4.2 Statistical Summary (`describe`)

The second check is semantic: do the numbers make physical sense? The `df.describe()` method calculates summary statistics (mean, standard deviation, min, max) for every numerical column.

```
print(df.describe().T) # .T transposes the output for readability
```

Listing 8: Summary Statistics

Column	Mean	Min	Max	...
year	1991.5	1958.0	2024.0	...
average	355.1	313.0	425.0	...
sdev	**-3.5**	**-9.99**	0.8	...
unc	**-0.8**	**-0.99**	0.5	...

Table 1: Simplified output of `describe()`

Sentinel Values
Look closely at the `min` value for the `sdev` column: **-9.99**. Physical standard deviation cannot be negative. This indicates that the dataset uses **Sentinel Values**.

- **Definition:** A sentinel value is a specific number (often -1, -99, or -999) used to represent "missing data" in systems that cannot handle empty fields (NaN).
- **The Danger:** If we blindly fed this column into a regression model, the model would treat -9.99 as a valid measurement. It would try to fit the curve to these negative values, pulling the entire function down and ruining the fit.

The `describe()` function allows us to spot these anomalies instantly because the `min` value is vastly different from the `mean` or `50%` (median) quartiles.

2.4.3 Filtering Invalid Data

Having identified the sentinel values, we must filter them out before training. We see from the dataset documentation (and the statistical summary) that missing values are marked with numbers less than 0 in the `sdev` column or `unc` column.

However, for our specific goal (fitting the trend of CO_2) we are primarily interested in the `average` column. We must ensure `average` is valid.

```python
# Check for common missing-value sentinels in the main column
min_val = df['average'].min()
print(f"Minimum CO2 value: {min_val}")

if min_val < 0:
    print("Warning: Sentinel values detected in target variable!")
```

Listing 9: Filtering Missing Values

3 The Design Matrix (Φ)

Now that we possess a vector of inputs $\mathbf{x}$ (years) and a vector of targets $\mathbf{t}$ (CO_2 levels). Our goal is to fit a polynomial curve to this data. However, standard linear algebra solvers (like those found in NumPy) are designed to solve *systems of linear equations*. So how do we use a "straight-line" tool to draw a "curved" prediction? The answer lies in **Feature Engineering**, specifically the transformation of inputs using *Basis Functions*.

3.1 Transforming Inputs to Features

3.1.1 The Limitation of Raw Inputs

Let us assume the simplest possible linear model, where we try to predict the target t directly from the raw input x:

$$y(x, \mathbf{w}) = w_0 + w_1 x \tag{8}$$

Geometrically, this equation describes a straight line. w_0 is the intercept, and w_1 is the slope.

If we look at the Mauna Loa data (Figure 2), it is clearly curved; the rate of CO_2 increase is accelerating (concave up). A straight line $w_0 + w_1 x$ will effectively "underfit" this data. To capture curvature, we need terms like x^2 (quadratic) or x^3 (cubic).

3.1.2 The "Basis Function" Trick

Instead of changing the *algorithm* (Linear Regression), we change the *data*. We transform our single input scalar x into a vector of features $\boldsymbol{\phi}(x)$. We define a set of functions called **Basis Functions**, denoted as $\phi_j(x)$.

$$y(x, \mathbf{w}) = \sum_{j=0}^{M} w_j \phi_j(x) = \mathbf{w}^T \boldsymbol{\phi}(x) \tag{9}$$

where $\mathbf{w} = [w_0, \ldots, w_M]^T$ and $\boldsymbol{\phi}(x) = [\phi_0(x), \ldots, \phi_M(x)]^T$.

3.1.3 The Polynomial Basis

For polynomial curve fitting, we choose a specific set of basis functions: the powers of x.

$$\phi_j(x) = x^j \tag{10}$$

This gives us the feature vector:

$$\boldsymbol{\phi}(x) = \begin{bmatrix} 1 \\ x \\ x^2 \\ \vdots \\ x^M \end{bmatrix} \tag{11}$$

Note that $\phi_0(x) = x^0 = 1$, which corresponds to the bias term (intercept) w_0.

Why $[1, x, x^2, \dots]$? By transforming the scalar x into this vector, we convert the non-linear problem:

$$y = w_0 + w_1 x + w_2 x^2$$

into a linear problem (dot product):

$$y = \mathbf{w} \cdot \mathbf{v}$$

where $\mathbf{v}$ is our new feature vector $[1, x, x^2]$.

3.2 The Design Matrix

In the previous section, we defined the feature vector $\boldsymbol{\phi}(x)$ for a single input x. However, our dataset consists of N observations: $\mathbf{x} = \{x_1, x_2, \dots, x_N\}^T$. To perform efficient computations, we must organize all these feature vectors into a single matrix, known as the **Design Matrix** (denoted by the capital Greek letter Φ).

3.2.1 The Mathematical Structure

The Design Matrix Φ is constructed by stacking the feature vectors of all N data points as rows.

$$\Phi = \begin{pmatrix} \boldsymbol{\phi}(x_1)^T \\ \boldsymbol{\phi}(x_2)^T \\ \vdots \\ \boldsymbol{\phi}(x_N)^T \end{pmatrix} \tag{12}$$

Explicitly, Φ is an $N \times (M+1)$ matrix:

$$\Phi = \begin{pmatrix} 1 & x_1 & x_1^2 & \dots & x_1^M \\ 1 & x_2 & x_2^2 & \dots & x_2^M \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & x_N & x_N^2 & \dots & x_N^M \end{pmatrix} \tag{13}$$

This specific structure—where each column is a power of the input vector—is known in linear algebra as a **Vandermonde Matrix**. It is the standard basis for polynomial interpolation (Golub & Van Loan, 2013).

With this matrix defined, we can write the model prediction for the entire dataset as a single matrix-vector multiplication:

$$\mathbf{y} = \Phi \mathbf{w} \tag{14}$$

where $\mathbf{y}$ is the $N \times 1$ vector of predictions, and $\mathbf{w}$ is the $(M+1) \times 1$ vector of coefficients.

3.2.2 Python Implementation

While NumPy has a built-in function `np.vander` that generates this matrix, we will implement it manually to fully understand the mechanics of feature engineering. The challenge in Python is to transform our 1D array of inputs `x` (shape `(N,)`) into this 2D matrix Φ (shape `(N, M+1)`).

```python
import numpy as np

def design_matrix(x, M):
    """
    Constructs the Vandermonde matrix Phi.
    Args:
        x: Input vector of shape (N,)
        M: Polynomial degree (int)

    Returns:
        Phi: Design matrix of shape (N, M+1)
    """
    # 1. List Comprehension: Create a list of arrays [x^0, x^1, ..., x^M]
    feature_list = [x**j for j in range(M + 1)]

    # 2. Stack them vertically
    stacked = np.vstack(feature_list)

    # 3. Transpose to get correct shape
    Phi = stacked.T

    return Phi
```

Listing 10: Constructing the Design Matrix

Why `.vstack` and `.T`? Let us trace the shapes step-by-step for a simple example where $N = 10$ (data points) and $M = 2$ (quadratic).

1. **Input `x`:** Shape is `(10,)`.
2. **List Comprehension:**
   - `j=0`: `x**0` produces an array of ones. Shape `(10,)`.
   - `j=1`: `x**1` is the original x. Shape `(10,)`.
   - `j=2`: `x**2` is x squared. Shape `(10,)`.

   We now have a Python list containing 3 arrays, each of length 10.
3. **`np.vstack(feature_list)`:** This function stacks arrays *vertically* (row by row).
   - Row 0 becomes x^0.
   - Row 1 becomes x^1.
   - Row 2 becomes x^2.

   The resulting matrix has shape `(3, 10)`. This is an $(M + 1) \times N$ matrix.
4. **`stacked.T` (Transpose):** Our mathematical definition of Φ requires data points to be rows (N) and features to be columns ($M + 1$). We apply the transpose operation `.T` to flip the dimensions from `(3, 10)` to `(10, 3)`.

3.3 The Problem of Scale

In Section 3.2, we constructed the design matrix Φ mathematically. However, if we were to apply this matrix directly to the Mauna Loa dataset using the raw years (e.g., $x = 1958.2$), our computer would fail to find a good solution. This failure is due to a fundamental limitation of how computers represent numbers. This phenomenon is known as **Numerical Instability** or **Ill-Conditioning**.

3.3.1 The Exploding Powers Phenomenon

Recall that our inputs x represent years, roughly in the range $[1958, 2025]$. Let us compute the values in the design matrix for a polynomial of degree $M = 10$.

- **Column 0 (Bias):** $x^0 = 1$
- **Column 1 (Linear):** $x^1 \approx 2 \times 10^3$
- **Column 10 (High Degree):** $x^{10} \approx (2 \times 10^3)^{10} = 2^{10} \times 10^{30} \approx 10^{33}$

Our design matrix Φ now contains numbers ranging from 1 to 10^{33} in the same row.

The "Squaring" Effect: The solution to the least squares problem involves the matrix $\Phi^T\Phi$. This operation effectively squares the eigenvalues of the matrix.

$$(x^{10})^2 = x^{20} \approx 10^{66} \tag{15}$$

While modern 64-bit floating-point numbers (`float64`) can store numbers up to $\approx 10^{308}$, the problem is not *storage* (overflow), but *precision* (significance).

3.3.2 IEEE 754 and Floating-Point Precision

Computers use the **IEEE 754** standard to represent real numbers (Goldberg, 1991). A `float64` number has 53 bits of significand (precision), which translates to approximately 15–17 significant decimal digits.

This means a computer cannot distinguish between:

$$1.0000000000000000$$

and

$$1.0000000000000001$$

When we perform matrix operations (like inversion or solving linear systems) on a matrix containing both 1 and 10^{66}, the small numbers effectively vanish or become dominated by the rounding errors of the large numbers.

3.3.3 The Condition Number

In numerical linear algebra, this sensitivity to scale is measured by the **Condition Number**, denoted $\kappa(A)$.

$$\kappa(A) = ||A|| \cdot ||A^{-1}|| \tag{16}$$

A low condition number (near 1) means the matrix is "well-behaved." A high condition number (e.g., 10^{15}) means the matrix is "ill-conditioned."

For the Vandermonde matrix constructed from raw years (1958–2025), the condition number is astronomically high. Consequently, when we try to solve for the weights $\mathbf{w}$, the computer's rounding errors are amplified by this factor of 10^{15} or more, rendering the calculated weights completely garbage (Higham, 2002).

The Solution: To fix this, we must shift the "ruler." We need to rescale our inputs x so that they all fall within a small, manageable range (typically near 0 and 1). This process is called **Standardization**, which we cover in the next section.

3.4 Standardization (Z-Score Normalization)

To solve the problem of exploding powers ($x^{20} \approx 10^{66}$), we must rescale our input data. We want to transform the raw years (e.g., 1958–2025) into a range where the numbers are small, typically centered around zero with a spread of roughly 1. The standard technique for this is **Z-Score Normalization** (or simply *Standardization*).

3.4.1 The Mathematical Transformation

For a given feature vector $\mathbf{x}$, we calculate two statistics:

1. The **Mean** (μ): The average value.
2. The **Standard Deviation** (σ): A measure of how spread out the values are.

We then transform every data point x_i into a standardized value z_i:

$$z_i = \frac{x_i - \mu}{\sigma} \tag{17}$$

Properties of the Standardized Data (z): After this transformation:

- The mean of z is exactly 0.
- The standard deviation of z is exactly 1.
- The values typically fall within the range $[-2, 2]$ or $[-3, 3]$.

Now, if we compute z^{20}, the values remain manageable. For example, $2^{20} \approx 10^6$, which fits comfortably within standard floating-point precision, solving the condition number problem (Hastie et al., 2009).

3.4.2 Avoiding Data Leakage

A critical conceptual error often made by beginners is calculating μ and σ using the *entire* dataset (Training + Test).

> **The Golden Rule:** You must compute scaling statistics (μ, σ) using **only** the training set. You then apply those specific values to transform the validation and test sets.

Why? In a real-world scenario (like predicting CO_2 in the year 2050), we do not know the future data. If we calculate the mean using data from 2050, we have allowed information from the future to "leak" into our model training. This is called **Data Leakage**. It makes the model perform artificially well during testing but fail in production (Kaufman et al., 2012).

3.4.3 Python Implementation and Broadcasting

We implement a function that calculates statistics on the training set and applies them to any other datasets passed to it.

```python
import numpy as np

def standardize(train_x, *others):
    """
    Standardize data using training set statistics.

    Args:
        train_x: The training data (used to compute mu and sigma).
        *others: Variable list of other datasets (val, test) to transform.

    Returns:
        transformed_train, transformed_others, mu, sigma
    """
    # 1. Compute statistics ONLY on training data
    mu = train_x.mean()
    sig = train_x.std()

    # 2. Transform training data
    train_z = (train_x - mu) / sig

    # 3. Transform other datasets using the SAME mu and sig
    outs = []
    for x in others:
        z = (x - mu) / sig
        outs.append(z)

    return train_z, outs, mu, sig
```

Listing 11: Standardization Function

Broadcasting Consider the line:

```python
train_z = (train_x - mu) / sig
```

- `train_x` is a NumPy array of shape `(N,)`.
- `mu` is a scalar (single float).

Mathematically, you cannot subtract a scalar from a vector. However, NumPy uses a mechanism called **Broadcasting**. It conceptually "stretches" the scalar `mu` into an array of shape `(N,)` consisting of repeated copies of μ, so that the shapes match. It then performs the subtraction element-by-element.

This operation is highly optimized in C, avoiding the need for slow Python loops like:

```python
# SLOW (Don't do this):
z = []
for val in train_x:
    z.append((val - mu) / sig)
```

4 The Least Squares Solution

We have constructed a Design Matrix Φ (containing our standardized features) and a target vector $\mathbf{t}$ (containing the CO_2 measurements). We established a linear model:

$$\mathbf{y} = \Phi\mathbf{w} \tag{18}$$

Our goal is to find the specific vector of weights $\mathbf{w}$ that makes the predictions $\mathbf{y}$ as close as possible to the targets $\mathbf{t}$.

To do this, we must mathematically define what "close" means. We need a quantitative measure of the quality of our model, known as an **Objective Function**, **Cost Function**, or **Loss Function**.

4.1 The Loss Function

4.1.1 Defining the Residual

For a single data point n, the difference between the model's prediction and the actual observed value is called the *residual* (or error), denoted by e_n:

$$e_n = y(x_n, \mathbf{w}) - t_n = \mathbf{w}^T\boldsymbol{\phi}(x_n) - t_n \tag{19}$$

If $e_n = 0$, the model predicts the data point perfectly. If $e_n > 0$, the model overestimates; if $e_n < 0$, it underestimates.

4.1.2 Sum of Squared Errors

We need to aggregate these N individual residuals into a single scalar value representing the total error of the model. A naive approach might be to simply sum the residuals: $\sum e_n$. However, this is flawed because positive errors (overestimates) and negative errors (underestimates) would cancel each other out, potentially resulting in a total error of zero for a terrible model.

Instead, we square the residuals to ensure they are always positive. This yields the **Sum of Squared Errors (SSE)**:

$$E(\mathbf{w}) = \frac{1}{2}\sum_{n=1}^{N}\{\mathbf{w}^T\boldsymbol{\phi}(x_n) - t_n\}^2 \tag{20}$$

Note on the Factor of $\frac{1}{2}$: You will often see a factor of $\frac{1}{2}$ included in the definition. This is purely for mathematical convenience. When we take the derivative of the squared term later ($\frac{d}{dw}w^2 = 2w$), the 2 from the exponent will cancel with the $\frac{1}{2}$, leaving a clean equation. It does not change the location of the minimum (Bishop, 2006).

4.1.3 Euclidean Norm

We can express the SSE compactly using the **Squared Euclidean Norm** (L_2 norm):

$$E(\mathbf{w}) = \frac{1}{2}||\Phi\mathbf{w} - \mathbf{t}||^2 \tag{21}$$

where $||\mathbf{v}||^2 = \mathbf{v}^T\mathbf{v} = v_1^2 + v_2^2 + \cdots + v_N^2$.

This vector notation allow us to use the rules of Matrix Calculus to solve for $\mathbf{w}$ analytically.

4.1.4 The Gaussian Noise Assumption

Why do we choose the *square* of the error? Why not the absolute value $|y - t|$? Or the fourth power $(y-t)^4$? The choice of Least Squares is not arbitrary; it is derived directly from the **Maximum Likelihood Principle** under the assumption of Gaussian noise.

Recall our data generation model from Section 1:

$$t = y(x, \mathbf{w}) + \epsilon, \quad \text{where } \epsilon \sim \mathcal{N}(0, \sigma^2) \tag{22}$$

If the noise ϵ follows a Gaussian distribution, the probability of observing a specific target t given input x is:

$$p(t|x, \mathbf{w}, \sigma) = \frac{1}{\sqrt{2\pi\sigma^2}} \exp\left(-\frac{(y(x, \mathbf{w}) - t)^2}{2\sigma^2}\right) \tag{23}$$

Maximizing Likelihood = Minimizing Squares: We want to find the weights $\mathbf{w}$ that maximize the probability (likelihood) of observing our entire training dataset.

- To maximize the exponential term $\exp(-\text{Error}^2)$, we must **minimize** the negative term in the exponent.
- The term in the exponent is exactly proportional to $(y - t)^2$.

Therefore, minimizing the Sum of Squared Errors is statistically equivalent to finding the most probable parameters, assuming the data has Gaussian measurement noise.

4.2 Deriving the Normal Equation

We have defined the error function $E(\mathbf{w})$ as a convex paraboloid (a bowl shape) and the minimum of a smooth function occurs where its derivative (slope) is zero. Since $\mathbf{w}$ is a vector of parameters, we must take the **Gradient** ($\nabla_{\mathbf{w}}$), which is the vector of partial derivatives with respect to each weight w_j.

$$\nabla E(\mathbf{w}) = \begin{pmatrix} \frac{\partial E}{\partial w_0} \\ \vdots \\ \frac{\partial E}{\partial w_M} \end{pmatrix} = \mathbf{0} \tag{24}$$

To find this gradient, we first expand the vector norm definition of the loss function.

4.2.1 Expanding the Quadratic Form

Recall the squared Euclidean norm definition: $||\mathbf{v}||^2 = \mathbf{v}^T\mathbf{v}$. Let $\mathbf{v} = \Phi\mathbf{w} - \mathbf{t}$ be the residual vector.

$$E(\mathbf{w}) = \frac{1}{2}(\Phi\mathbf{w} - \mathbf{t})^T(\Phi\mathbf{w} - \mathbf{t}) \tag{25}$$

We apply the matrix transpose identity $(A + B)^T = A^T + B^T$ to the first term:

$$(\Phi\mathbf{w} - \mathbf{t})^T = (\Phi\mathbf{w})^T - \mathbf{t}^T = \mathbf{w}^T\Phi^T - \mathbf{t}^T \tag{26}$$

Note: The transpose of a product reverses the order: $(AB)^T = B^TA^T$.

Now, we substitute this back and distribute the terms:

$$E(\mathbf{w}) = \frac{1}{2}(\mathbf{w}^T\Phi^T - \mathbf{t}^T)(\Phi\mathbf{w} - \mathbf{t}) \tag{27}$$

$$= \frac{1}{2}\left(\mathbf{w}^T\Phi^T\Phi\mathbf{w} - \mathbf{w}^T\Phi^T\mathbf{t} - \mathbf{t}^T\Phi\mathbf{w} + \mathbf{t}^T\mathbf{t}\right) \tag{28}$$

4.2.2 Simplifying Scalar Terms

Look closely at the two middle terms: $\mathbf{w}^T\Phi^T\mathbf{t}$ and $\mathbf{t}^T\Phi\mathbf{w}$.

- $\mathbf{w}$ is $(M+1) \times 1$.
- Φ^T is $(M+1) \times N$.
- $\mathbf{t}$ is $N \times 1$.

The result of $\mathbf{w}^T\Phi^T\mathbf{t}$ is a (1×1) scalar. The same applies to $\mathbf{t}^T\Phi\mathbf{w}$. Since the transpose of a scalar is itself ($a^T = a$), and $(\mathbf{w}^T\Phi^T\mathbf{t})^T = \mathbf{t}^T\Phi\mathbf{w}$, these two terms are **identical**.

We can combine them:

$$E(\mathbf{w}) = \frac{1}{2}\mathbf{w}^T(\Phi^T\Phi)\mathbf{w} - \mathbf{w}^T(\Phi^T\mathbf{t}) + \frac{1}{2}\mathbf{t}^T\mathbf{t} \tag{29}$$

4.2.3 Computing the Gradient

Now we differentiate with respect to vector $\mathbf{w}$. We use two standard results from Matrix Calculus (Magnus & Neudecker, 2019):

Rule A (Linear Term): The derivative of $\mathbf{w}^T\mathbf{a}$ with respect to $\mathbf{w}$ is the vector $\mathbf{a}$.

$$\frac{\partial}{\partial\mathbf{w}}(\mathbf{w}^T\mathbf{a}) = \mathbf{a} \tag{30}$$

Here, $\mathbf{a} = \Phi^T\mathbf{t}$. So, the derivative of the second term is $-\Phi^T\mathbf{t}$.

Rule B (Quadratic Term): The derivative of the quadratic form $\mathbf{w}^TA\mathbf{w}$ (where symmetric $A = \Phi^T\Phi$) is $2A\mathbf{w}$.

$$\frac{\partial}{\partial\mathbf{w}}(\mathbf{w}^TA\mathbf{w}) = (A + A^T)\mathbf{w} = 2A\mathbf{w} \tag{31}$$

So, the derivative of the first term is $\frac{1}{2}(2\Phi^T\Phi\mathbf{w}) = \Phi^T\Phi\mathbf{w}$.

The third term ($\frac{1}{2}\mathbf{t}^T\mathbf{t}$) does not contain $\mathbf{w}$, so its derivative is zero.

4.2.4 Solving for Zero

Combining these results gives the gradient:

$$\nabla E(\mathbf{w}) = \Phi^T\Phi\mathbf{w} - \Phi^T\mathbf{t} \tag{32}$$

We set this to zero to find the minimum:

$$\Phi^T\Phi\mathbf{w} - \Phi^T\mathbf{t} = 0 \tag{33}$$

Rearranging gives the famous N**ormal Equations**:

$$(\Phi^T\Phi)\mathbf{w} = \Phi^T\mathbf{t} \tag{34}$$

Finally, assuming the matrix $\Phi^T\Phi$ is invertible, we multiply both sides by $(\Phi^T\Phi)^{-1}$ to isolate $\mathbf{w}$:

$$\mathbf{w}^* = (\Phi^T\Phi)^{-1}\Phi^T\mathbf{t} \tag{35}$$

This equation gives us the optimal weights $\mathbf{w}^*$ in a single step, without any iterative guessing or loops. It is the **Closed-Form Solution** to the Least Squares problem.

4.3 Numerical Stability in Implementation

A novice programmer might look at equation (35) and immediately translate it into Python using `numpy.linalg.inv`:

```
# DO NOT USE THIS IN PRODUCTION
Phi_T_Phi = Phi.T @ Phi
inverse = np.linalg.inv(Phi_T_Phi)
w = inverse @ Phi.T @ t
```

Listing 12: The Naive (and Dangerous) Implementation

While this code is mathematically correct, it is **numerically disastrous**. In professional scientific computing, explicitly inverting a matrix is considered a "cardinal sin" (Trefethen & Bau III, 1997).

4.3.1 The Dangers of the Explicit Inverse

The problem lies in the term $\Phi^T\Phi$. Recall the concept of the **Condition Number** κ from Section 3.3.3. The condition number measures how much errors in the input are amplified in the output. When we multiply Φ^T by Φ, we square the condition number:

$$\kappa(\Phi^T\Phi) = (\kappa(\Phi))^2 \tag{36}$$

Example: Suppose our design matrix Φ has a condition number of 10^7. This is high, but manageable for double-precision floats (which handle up to $\approx 10^{15}$). However, the matrix $\Phi^T\Phi$ will have a condition number of $(10^7)^2 = 10^{14}$. This is dangerously close to the machine epsilon limit (10^{16}). Any small noise in the data or rounding errors in the calculation will be amplified by a factor of $100,000,000,000,000$. The resulting weights $\mathbf{w}$ will be garbage.

4.3.2 Singular Value Decomposition (SVD)

To solve this, numerical linear algebra libraries (like LAPACK, which powers NumPy) use a technique that solves for $\mathbf{w}$ *without ever forming the matrix* $\Phi^T\Phi$.

The standard approach is **Singular Value Decomposition (SVD)**. SVD breaks any matrix Φ into three simpler matrices:

$$\Phi = U\Sigma V^T \tag{37}$$

- U and V are orthogonal matrices (rotations), which are numerically very stable.
- Σ is a diagonal matrix containing the **Singular Values** (scales).

By substituting this into the normal equation, the solution simplifies to:

$$\mathbf{w}^* = V\Sigma^{-1}U^T\mathbf{t} \tag{38}$$

Crucially, calculating Σ^{-1} only requires dividing by the singular values ($1/\sigma_i$). We never square them. Therefore, the condition number stays at 10^7, not 10^{14}. This method is twice as precise as the explicit inverse.

4.3.3 `np.linalg.lstsq`

In Python, we access this stable SVD solver via the function `numpy.linalg.lstsq` (Least Squares).

```python
def fit_least_squares(Phi, t):
    """
    Solves w = argmin ||Phi*w - t||^2 using SVD.
    """
    # rcond=None lets NumPy determine the cutoff for small singular values
    w, residuals, rank, s = np.linalg.lstsq(Phi, t, rcond=None)
    return w
```

Listing 13: The Robust Implementation

Parameter Explanation:

- `Phi`: The design matrix ($N \times M$).
- `t`: The target vector ($N \times 1$).
- `rcond`: The "Reciprocal Condition Number." This is a cutoff threshold. If a singular value σ_i is extremely small (indicating that a feature is redundant or the matrix is near-singular), SVD treats it as zero rather than trying to divide by it (which would result in infinity). This makes the solver robust even if we have "bad" data.

Return Values: The function returns four values, but we usually only care about the first one:

1. `w`: The optimal weights.
2. `residuals`: The squared error ($E(\mathbf{w})$) of the solution.
3. `rank`: The effective dimensionality of the data.
4. `s`: The singular values themselves.

References

Bishop, C. M. (2006). *Pattern recognition and machine learning*. Springer.

Friedl, J. E. F. (2006). *Mastering regular expressions*. O'Reilly Media, Inc. (The standard reference for understanding regex patterns like '+'.)

Goldberg, D. (1991). What every computer scientist should know about floating-point arithmetic. *ACM Computing Surveys (CSUR)*, *23*(1), 5–48. (Explains why computers cannot do simple math like 0.1 + 0.2 == 0.3 correctly.)

Golub, G. H., & Van Loan, C. F. (2013). *Matrix computations* (4th ed.). Johns Hopkins University Press. (The bible of numerical linear algebra, detailing the properties and stability issues of Vandermonde matrices.)

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., et al. (2020). Array programming with numpy. *Nature*, *585*(7825), 357–362. (The definitive paper describing the internal architecture of NumPy.)

Hastie, T., Tibshirani, R., & Friedman, J. (2009). *The elements of statistical learning: Data mining, inference, and prediction*. Springer Science & Business Media. (Focuses on the bias-variance tradeoff and the mathematical underpinnings of regression.)

Higham, N. J. (2002). *Accuracy and stability of numerical algorithms* (2nd ed.). SIAM. (The advanced text on why matrix inversion fails for ill-conditioned matrices.)

Kaufman, S., Rosset, S., Perlich, C., & Stitelman, O. (2012). Leakage in data mining: Formulation, detection, and avoidance. *ACM Transactions on Knowledge Discovery from Data (TKDD)*, *6*(4), 1–21. (A comprehensive definition of data leakage (using test information during training).)

Keeling, C. D., Bacastow, R. B., Bainbridge, A. E., Ekdahl, C. A., Guenther, P. R., Waterman, L. S., & Chin, J. F. S. (1976). Atmospheric carbon dioxide variations at mauna loa observatory, hawaii. *Tellus*, *28*(6), 538–551.

Magnus, J. R., & Neudecker, H. (2019). *Matrix differential calculus with applications in statistics and econometrics* (3rd ed.). John Wiley & Sons. (The definitive reference for deriving gradients of vector/matrix operations.)

McKinney, W. (2010). Data structures for statistical computing in python. In *Proceedings of the 9th python in science conference* (Vol. 445, pp. 51–56). (The original introduction of Pandas.)

Murphy, K. P. (2012). *Machine learning: A probabilistic perspective*. MIT press. (Provides deep insights into Gaussian noise assumptions and probabilistic modelling.)

Strang, G. (2006). *Linear algebra and its applications*. Belmont, CA: Thomson, Brooks/Cole. (The definitive guide to understanding the systems of linear equations ($Ax = b$) that underpin regression.)

Trefethen, L. N., & Bau III, D. (1997). *Numerical linear algebra*. SIAM. (The gold standard textbook for understanding why SVD is better than matrix inversion.)

Tukey, J. W. (1977). *Exploratory data analysis*. Reading, Mass.: Addison-Wesley. (The classic text establishing the philosophy of looking at data before modeling it.)

VanderPlas, J. (2016). *Python data science handbook: Essential tools for working with data*. O'Reilly Media, Inc. (An excellent resource for understanding NumPy vectorization and Matplotlib.)